

informe_completo

Medición del 9 de octubre de 2026 · sistema de geolocalización por visión monocular · LAC, PUC-Rio

9 de octubre de 2026

0. Para qué sirve este documento

Hay tres lectores posibles y el texto sirve a los tres sin cambiar de registro.

El primero sos vos dentro de seis meses, cuando no te acuerdes por qué la Pi 4 quedó configurada a una imagen por segundo. El segundo es Bruno y el grupo, que hicieron preguntas concretas el 2 de octubre y merecen respuestas con números. El tercero es quien te entrevistó para un puesto de *Edge AI* o *Perception Engineer*, que va a querer ver no el resultado sino **cómo llegaste a él**.

Por eso cada número viene con su cuenta desarrollada. Un número suelto se olvida y no se puede defender; una cuenta se reconstruye.

1. Qué pidió el grupo, textual

De la reunión del 2 de octubre, *Evolução do Sistema BeeSwarm*. Cito literal y traduzco a qué significa en términos de trabajo.

1.1 Medir cuánto cuesta

"Vamos ver o quanto isso pesa em processamento. Aí nós sentamos com o resultado e vemos o próximo passo."

"vamos rodar o que a gente tem hoje"

Qué significa. Es un pedido de medición, no de funcionalidad. "Correr lo que tenemos hoy" es explícito: no agregues capacidades, cuantifícalas que hay. Este documento es la respuesta.

1.2 Explicar la anomalía de los FPS

"Eu tô vendo que lá em cima tá dando 4 FPSs. Esse é uma média entre os dois Raspberries ou é... cada um tá conseguindo 4 FPSs?"

"Uai, o Raspberry 4 e o 5 tão dando a mesma coisa? **Não faz sentido.**"

"**A gente tem que saber explicar por que que funcionou.**"

Qué significa. Bruno detectó una inconsistencia real y nadie supo explicarla ese día. Tenía razón en sospechar: dos computadoras con una generación de diferencia no pueden rendir igual. La respuesta está en la sección 5 y es que **la pantalla mostraba la tasa pedida, no la entregada**. Eran dos números distintos con el mismo nombre.

1.3 La duda sobre el vector de apariencia

Planteada dos veces, la segunda citando que Bruno ya la había hecho antes:

"A gente passou um bounding box, a gente deu uma cropada e a gente fez o vetor. Mas na hora de comparação do vetor... se você ver uma imagem de um lado e ver uma imagem do outro... elas tão invertidas, **uma pessoa vista da direita, uma pessoa vista da esquerda. É a mesma pessoa. Mas a imagem é invertida.**"

"é a mesma dúvida que o Bruno tinha tido do vetor de embedding pra comparar diferentes detecções... **isso pode variar muito dependendo da direção**... não era melhor tipo usar meio que a geoloca...?"

La respuesta que diste en la reunión fue correcta pero cualitativa:

"É uma heurística juntada com isso, então é... **é os dois**, né?"

Qué significa. Bruno está señalando el problema de **invariancia de punto de vista** en re-identificación. Y tiene razón empírica: el número está medido y es incómodo. Lo desarrollo en la sección 3.4.

1.4 Dos drones, y esperar antes de integrar

"Eu só tenho uma sugestão aqui. **Vamos trabalhar só com dois drones por enquanto**, Sebastian? Porque a gente nem tem o terceiro com câmera direito."

"vamos esperar um pouquinho antes de botar as coisas na nossa ground station"

"**Espera mais uma semana ou duas** pra gente ter essa conversa. Vamos ver isso rodando no mundo real."

Qué significa. Dos aeronaves, no tres. La integración con la estación del grupo queda postergada **por decisión de ellos**, no por falta tuya. Y hay una fecha: una o dos semanas desde el 2 de octubre vencen entre el 9 y el 16.

2. Qué es Edge AI y qué se mide de verdad

Tu pregunta de fondo: ¿es solo latencia y capacidad del modelo? No. Son **siete ejes**, y el error más común es medir uno y declarar el sistema.

2.1 Los siete ejes

(1) Tasa sostenida, no de ráfaga. Cuántas inferencias por segundo puede mantener el sistema **durante el tiempo que dura la misión**, no durante treinta segundos. Es el eje que más se falsea, porque medir corto es barato y da números buenos. La sección 5.3 muestra el caso exacto: a los dos minutos la Pi 4 parecía aprobada y a los ocho estaba frenándose.

(2) Latencia por etapa, con percentiles. La tasa dice *cada cuánto* produce el sistema. La latencia dice *qué tan vieja* es cada respuesta. Son independientes: un sistema puede entregar a 10 Hz con 3 segundos de retraso. Y hay que medirla **por etapa**, porque el total no dice dónde optimizar.

(3) Energía por inferencia. En joules por cuadro. Es lo que traduce una decisión de cómputo en minutos de autonomía, que en un dron es la restricción que manda sobre todas las demás.

- (4) **Memoria y comportamiento bajo presión.** No solo cuánta usa, sino qué pasa cuando falta. Si el sistema entra en memoria de intercambio, **la tasa media casi no se mueve** mientras la latencia individual se dispara. Es un fallo que solo se ve mirando la cola.
- (5) **Precisión en cada configuración.** Toda optimización de velocidad paga en algo. La pregunta no es "¿cuánto más rápido?" sino "¿cuánto más rápido **y a qué costo de detección?**". Esto exige verdad de campo, que es caro de conseguir, y por eso casi nadie lo publica.
- (6) **La cola, no el promedio.** El percentil 95 y el 99. Un sistema rápido en diecinueve cuadros de veinte y trabado en el veinte tiene un promedio excelente y pierde objetivos. El promedio es exactamente el estadístico que esconde este fallo.
- (7) **Las decisiones de modelo y de runtime.** Cuantización (INT8 contra FP32), resolución de entrada, número de hilos, motor de inferencia (NCNN, TFLite, ONNX Runtime). Estas son las palancas, y solo sirven si sabés sobre qué eje querés mover.

2.2 Dónde estás parado

De los siete, tenés **cinco medidos**:

Eje	Estado	Dónde
1. Tasa sostenida	Medido	\$5.3, curva térmica de 8 minutos
2. Latencia por etapa	Medido	\$5.1, mediana de ciclo y de detector
3. Energía por inferencia	Falta	\$7.3
4. Memoria	Medido	\$5.4, 3,1 GB libres, sin intercambio
5. Precisión por configuración	Medido	\$5.5, sobre vuelo etiquetado
6. La cola (p95)	Instrumentado, aún sin dato válido	\$7.2
7. Modelo y runtime	Parcial	NCNN elegido; sin barrido de cuantización

Cinco de siete, con verdad de campo real en el quinto, es **más de lo que la mayoría de los proyectos puede mostrar**. Los dos que faltan son justamente los que un entrevistador técnico va a preguntar, así que conviene tener la respuesta preparada: "*no está medido, y sé exactamente cómo medirlo*" es una respuesta sólida; improvisar no lo es.

2.3 Lo que no es Edge AI

Vale decirlo porque aclara el resto. **No** es entrenar modelos; eso pasa en la nube, antes. **No** es elegir la arquitectura con mejor mAP en un *benchmark*; esa elección se hace bajo restricciones que el benchmark no tiene. Edge AI es el trabajo de hacer que un modelo ya entrenado **cumpla un contrato bajo restricciones físicas**: tantos vatios, tantos grados, tantos milisegundos, tanta memoria, tanta batería. La ingeniería está en el contrato, no en el modelo.

3. Cómo es tu sistema

3.1 La cadena, de punta a punta

cámara	→	detector	→	seguidor	→	apariencia	→	rayo	→	suelo	→	identidad	→	reporte
ArduCam		YOLO		BoT-SORT		OSNet		pinhole		z=0		fusión		radio
1920×1080		ncnn		+ CMC		ReID								GrADyS

Qué hace cada etapa, con lo que entra y lo que sale:

Cámara. ArduCam Module 3 (IMX708) en el puerto CAM1 de la Raspberry, leída con `picamera2`. Calibrada con tablero: focal 1407,0 px, centro (945,7 / 547,1), error de reproyección **4,25 px sobre 111 fotos válidas**. Esto importa: la focal entra directamente en el cálculo del rayo, y un error ahí se propaga a metros en el suelo.

Detector. YOLO compilado a NCNN, que es el motor de inferencia pensado para ARM. Sale una lista de cajas con clase y confianza.

Seguidor. BoT-SORT con compensación de movimiento de cámara (CMC). Le asigna a cada caja un identificador que persiste entre cuadros. El CMC importa en un dron porque la cámara se mueve: sin él, el movimiento propio se confunde con movimiento del objetivo.

Apariencia. OSNet produce un vector por recorte. Es lo que permite decir "esta persona de ahora es la misma de hace veinte segundos" cuando el seguidor perdió la pista.

Rayo. El paso geométrico. Un píxel y la pose de la aeronave dan una semirrecta en el espacio.

Suelo. Esa semirrecta se corta con el plano horizontal a la altura declarada. El corte es la posición estimada del objetivo.

Identidad. Acumula impactos por pista, decide si una pista está madura, y fusiona pistas distintas que probablemente sean la misma persona.

Reporte. Un mensaje JSON por el plano de datos de GrADyS, difundido a quien escuche.

3.2 La geometría, con la cuenta

Del píxel al rayo. Con `(u, v)` el píxel, `(cx, cy)` el centro óptico y `f` la focal en píxeles, la dirección en el sistema de la cámara es

```
d_cam = normalizar( u - cx , v - cy , f )
```

Esa dirección se rota al sistema del mundo por el rumbo de la aeronave y el cabeceo del montaje (-55° en el vuelo de referencia), más alabeo y cabeceo del fuselaje cuando hay fuente de actitud.

Del rayo al suelo. Con origen `o` (la posición de la aeronave) y dirección `d`, el corte con el plano `z = z_suelo` está en

```
k = (z_suelo - o_z) / d_z      p = o + k * d
```

Si $d_z \geq 0$ el rayo apunta al horizonte o por encima y no hay corte: esa detección se descarta.

Por qué esto importa para entender el sistema. La precisión en el suelo depende del **rango inclinado**, no de la altura. A 35 m de altura y -55° de cabeceo, el rango inclinado es $35 / \sin(55^\circ) = 42,7$ m. Un error de rumbo de 1° desplaza el impacto en $42,7 * \tan(1^\circ) = 0,75$ m. Esa es la razón de que el radio de fusión no sea una constante.

3.3 El radio de fusión, que es una fórmula y no un número

```
fusion_radius_m = gps_sigma + slant_range * yaw_sigma
```

Dos términos, dos fuentes de error distintas. El primero es la incertidumbre de la posición de la aeronave, que no depende de a dónde mire. El segundo es la incertidumbre de rumbo **multiplicada por la distancia**: cuanto más lejos mira, más cuesta un grado de error.

Esto es lo que separa un parámetro derivado de una constante mágica. Todos los términos están en la telemetría, así que el radio se recalcula por detección en vez de ajustarse a mano.

No todos los parámetros son así, y conviene no mentir sobre eso. `MIN_MEASUREMENTS`, `DUTY_MIN`, `COOCURRENCIA_MIN` y el umbral del detector **no** se derivan de los datos: codifican el costo de equivocarse, que cambia según la misión. Un rescate tolera más falsos positivos que una inspección. Presentarlos como físicamente derivados sería falso.

3.4 La duda de Bruno, con el número

Bruno preguntó si el vector de apariencia aguanta el cambio de punto de vista. La medición sobre vectores promediados en ventanas de cuatro segundos dice:

Con el umbral puesto, el 35 % de los pares de personas DISTINTAS pasa el filtro de apariencia.

Leído de frente: **la apariencia sola no distingue personas**. Lo que sostiene el sistema es que **la distancia filtra primero**, y la apariencia refina dentro de lo que la distancia ya dejó pasar. Tu respuesta en la reunión, "*é os dois*", era correcta. Ahora tiene un número detrás.

Esto no invalida nada, pero sí invalida **una frase**: no se puede decir "la apariencia distingue personas". Se puede decir "la distancia separa candidatos y la apariencia decide entre los que quedan cerca".

4. Cómo se midió, y por qué así

El método es la parte defendible. Un resultado sin método es una anécdota.

4.1 Instrumentar la cadena, no cronometrarla desde afuera

Un cronómetro externo mide el intervalo entre reportes, que incluye las esperas. Lo que interesa es el trabajo. Por eso cada reporte que sale de la aeronave lleva ahora:

<code>lat_ciclo_p50_ms</code>	la mediana del ciclo completo de una mirada
<code>lat_ciclo_p95_ms</code>	el percentil 95 del mismo ciclo

lat_detector_p50_ms	la mediana del detector solo
lat_muestras	cuántas miradas entraron en el intervalo
mem_mb	memoria disponible

4.2 Percentil 95 y no promedio

Ya está argumentado en §2.1(6) y es la decisión metodológica más importante. El promedio esconde exactamente el fallo que importa.

4.3 Vaciar la ventana en cada reporte

Los acumuladores se vacían al publicar. Un percentil acumulado sobre toda la misión queda dominado por el arranque, cuando se están cargando el detector y el modelo de apariencia, y seguiría informando ese arranque minutos después de que la placa se asentó. El operador necesita saber qué está haciendo la aeronave **ahora**.

4.4 Reloj de pared, no el del simulador

El reloj del proveedor lo mueve la simulación cuando se reproduce un vuelo grabado, e informaría lo que un cuadro **debía** tardar. La pregunta es cuánto tardó, así que se usa `time.perf_counter()`.

4.5 Las dos placas, la misma tasa, al mismo tiempo

Lo que vale no es el techo de cada placa aislada sino **dónde divergen**. La Pi 5 plana en 47 °C mientras la Pi 4 trepa hasta frenarse es todo el argumento, y ese contraste solo existe si a las dos se les pidió lo mismo.

4.6 Diez minutos por punto

No es exceso de celo. Es lo que tarda la temperatura en mostrar su pendiente. La sección 5.3 muestra el caso en que dos minutos mienten.

4.7 La precisión, sin hardware

El vuelo etiquetado se vuelve a correr submuestreado: una de cada N imágenes. La tasa declarada **se deriva de la lista de cuadros**, no se configura aparte, de modo que no hay dos perillas que puedan desincronizarse. El vuelo se grabó a 1,64 imágenes por segundo, así que la única dirección honesta es hacia abajo: tasas mayores no se pueden simular porque esos cuadros no existen.

5. Resultados, con los cálculos desarrollados

5.1 Latencia: el número que explica todo lo demás

Veinte muestras, las dos placas a 0,5 imágenes por segundo, cámara y detector reales.

	Pi 5	Pi 4
mediana del ciclo	103,8 ms	721,1 ms

	Pi 5	Pi 4
rango observado	94 – 148 ms	680 – 764 ms
muestras	10	7
mediana del detector	103,8 ms	721,1 ms

$$721,1 \text{ ms} / 103,8 \text{ ms} = 6,95$$

La Pi 4 tarda casi siete veces más por imagen que la Pi 5.

Y el segundo dato de esa tabla es tan importante como el primero: **la mediana del detector es igual a la del ciclo completo**, en las dos placas. La proyección del rayo, el corte con el suelo, el seguidor y la capa de identidad **juntos no suman un milisegundo medible**. Si hay que optimizar algo, es el detector, y nada más. Eso descarta de un golpe cualquier trabajo de optimización sobre la geometría o la asociación.

5.2 El techo de cada placa, con la cuenta

Si cada imagen cuesta T segundos, el máximo sostenible es 1/T :

$$\begin{aligned} \text{Pi 5: } & 1 / 0,1038 \text{ s} = 9,63 \text{ img/s} \\ \text{Pi 4: } & 1 / 0,7211 \text{ s} = 1,39 \text{ img/s} \end{aligned}$$

Aquí está la respuesta a Bruno. La misión pedía 3 imágenes por segundo, lo que exige terminar cada ciclo en menos de $1/3 = 333 \text{ ms}$. La Pi 4 tarda 721. **No es que vaya lenta: es aritméticamente incapaz.** Los 0,78 que se midieron el 7 de octubre son ese techo de 1,39 castigado por el frenado térmico.

Y la pantalla decía 4 FPS para las dos porque mostraba **la tasa pedida**, no la entregada. Dos cantidades distintas con el mismo nombre en la misma casilla.

5.3 Por qué una se calienta y la otra no

El ciclo de trabajo es la fracción del tiempo que el procesador está ocupado. Pidiendo 0,5 imágenes por segundo, el período es 2000 ms:

$$\begin{aligned} \text{Pi 5: } & 103,8 \text{ ms} / 2000 \text{ ms} = 5,2 \% \\ \text{Pi 4: } & 721,1 \text{ ms} / 2000 \text{ ms} = 36,1 \% \end{aligned}$$

Siete veces más trabajo por unidad de tiempo, y por eso siete veces más calor disipado. No hace falta ninguna otra explicación: **una placa estabiliza en 47 °C y la otra no estabiliza.**

La curva térmica, y la trampa

A **una** imagen por segundo, ocho minutos de banco:

minuto	Pi 4	Pi 5
0,0	70,1 °C	51,2 °C
2,0	74,5 °C	51,8 °C

minuto	Pi 4	Pi 5
4,0	75,5 °C	53,5 °C
6,0	77,9 °C	52,0 °C
8,0	79,9 °C	52,0 °C

La Pi 5 es plana. La Pi 4 sube aproximadamente $(79,9 - 70,1) / 8 = 1,2$ °C por minuto **y no se aplana**. El límite blando de una Raspberry Pi 4 son 80 °C, donde el firmware empieza a bajar la frecuencia. Lo cruza a los ocho minutos. **Un vuelo dura quince o veinte.**

La trampa, que es la lección metodológica del día. A los dos minutos la Pi 4 marcaba 74,5 °C y entregaba el 100 % de lo que declaraba. Parecía aprobada. Una medición de treinta segundos habría dicho que funciona. **El rendimiento sostenido y el de ráfaga no son el mismo número**, y en un vuelo solo cuenta el primero.

La prueba dura, del firmware

Al apagar las placas:

```
Pi 5: temp=47,2 °C  throttled=0x0
Pi 4: temp=75,0 °C  throttled=0xe0000
```

0xe0000 son tres bits: 0x20000 frecuencia del ARM limitada, 0x40000 frenado ocurrido, 0x80000 límite térmico blando alcanzado. Los tres son bits de "ocurrió alguna vez", y están puestos. **Esto no es una inferencia desde la curva: la placa se frenó y lo dice ella.** La Pi 5 cerró limpia.

5.4 Memoria

```
Pi 5: 3190 MB disponibles de 4050   Pi 4: 3132 MB disponibles de 3797
```

Se usa MemAvailable, no MemFree, que son preguntas distintas. MemFree cuenta solo lo que nadie tiene tomado, y en un Linux con rato encendido siempre es poco porque el núcleo se queda con la caché de páginas; una placa sana con dos gigas de caché parecería a punto de morir. MemAvailable es la estimación del propio núcleo de lo que una asignación nueva realmente conseguiría.

Conclusión: la memoria no es un cuello de botella. No hay intercambio. El eje 4 queda cerrado.

5.5 Precisión contra tasa: el resultado contraintuitivo

Vuelo del 2 de agosto, submuestreado. Siete personas con etiqueta humana como verdad de campo.

imágenes/s	cuadros usados	personas encontradas	falsos positivos
1,64	1381	5 de 7	6
0,82	691	5 de 7	4
0,55	461	4 de 7	5
0,41	346	5 de 7	3

Usando una cuarta parte de los cuadros, el sistema encuentra a las mismas cinco personas y produce la mitad de los falsos positivos.

El mecanismo es plausible y vale explicarlo: menos cuadros son menos oportunidades para que una pista espuria acumule evidencia suficiente para madurar. Y como los umbrales de madurez se escalan por la tasa declarada, veinte segundos de evidencia siguen siendo veinte segundos reales de reloj, no menos.

Esto convierte el calor de la Pi 4 de bloqueante en parámetro. Si térmicamente solo aguanta media imagen por segundo, a esa tasa encuentra lo mismo.

5.6 Latencia contra error de posición: una cuenta que falta hacer

Un POI reportado con T segundos de retraso describe dónde estaba la persona hace T segundos. Con una persona caminando a v metros por segundo, el error introducido es $v \times T$.

A paso normal, $v = 1,4$ m/s:

Pi 5: $1,4 \text{ m/s} * 0,104 \text{ s} = 0,15 \text{ m}$
Pi 4: $1,4 \text{ m/s} * 0,721 \text{ s} = 1,01 \text{ m}$

El error mediano de todo el sistema es 2,39 m. La latencia de la Pi 4 aporta sola un metro de eso, un 42 % del presupuesto de error, **sobre objetivos en movimiento**. La de la Pi 5 aporta quince centímetros.

Esta cuenta no estaba hecha antes de hoy porque la latencia no se medía. Es el tipo de cosa que no se descubre razonando.

6. Qué significa para volar

La decisión ya no es una opinión:

La Pi 5 vuela a 3 imágenes por segundo. Entrega 2,93–3,26, temperatura plana en 47–53 °C, sin slots perdidos. Su techo es 9,6, así que está trabajando al 31 % de su capacidad. Hay margen de sobra.

La Pi 4 no vuela a 3. Es aritméticamente imposible. Y a 1 cruza el límite térmico a los ocho minutos.

Tres salidas, en orden de lo que yo haría:

(a) Un disipador o ventilador en la Pi 4. El problema es térmico y la solución de un problema térmico cuesta cinco dólares. Hay que probarlo antes de descartar la placa: resolvería el eje térmico y dejaría la placa en su techo de cómputo de 1,39.

(b) Volar la Pi 4 a 0,5 imágenes por segundo. Ciclo de trabajo del 36 % contra el 72 % que tendría a 1, y la sección 5.5 dice que no cuesta detecciones. Hay que medir si a esa tasa se estabiliza bajo 80.

(c) Volar un solo dron. El grupo pidió dos, pero un dron que cumple su contrato vale más que dos donde uno miente sobre lo que entrega.

7. Qué no sabemos

Esta sección existe porque un informe sin ella no es creíble.

7.1 La tabla de precisión es un vuelo y siete personas

Cuatro puntos, $n = 7$. El 4 de 7 a 0,55 img/s **es ruido**: con siete personas, la diferencia entre cuatro y cinco es una sola persona. Sirve para decidir hoy; no para afirmar que la relación vale en general. Para eso harían falta varios vuelos y más personas.

7.2 El percentil 95 todavía no significa nada

A media imagen por segundo entra **una sola mirada** en cada intervalo de reporte, de modo que el p95 y la mediana son literalmente la misma muestra. Los percentiles recién tienen sentido a 3 o 4 imágenes por segundo, donde entran seis u ocho miradas por intervalo. La instrumentación está lista; falta la corrida.

7.3 La energía no está medida

Falta el eje 3 completo. Se puede aproximar con la caída de tensión de la batería a lo largo de una corrida larga, que ya se está registrando en el CSV, pero una aproximación por tensión es mala: la curva de descarga de una LiPo es plana en el medio de su rango, que es justo donde se vuela. Lo correcto es un medidor de corriente en serie.

7.4 La Pi 4 a 0,5 img/s no se midió térmicamente

Se midió a 1. El barrido completo de cinco tasas se cortó por tiempo con veinte muestras. Es la corrida que falta para cerrar la tabla.

7.5 La SIYI no está calibrada

La cámara SIYI A8 mini quedó funcionando por RTSP el 8 de octubre, pero sus parámetros intrínsecos son **estimados de las especificaciones** (focal 1130 px calculada como $4,47 * 1920 / 7,6$), no medidos con tablero. La ArduCam sí está calibrada, con 4,25 px de error de reproyección. Esa diferencia se propaga directamente al error en metros, así que para geolocalizar hay que usar la ArduCam hasta que la SIYI se calibre.

8. Guión para Bruno

En portugués, que es como vas a hablarle. Pregunta por pregunta.

"Por que o Raspberry 4 e o 5 davam a mesma coisa? Não faz sentido."

Você estava certo, não fazia sentido. A tela mostrava a taxa **pedida**, não a entregue. Eram dois números diferentes com o mesmo nome.

Medimos o tempo real por imagem: o Pi 5 leva 104 ms, o Pi 4 leva 721 ms. Quase sete vezes mais. Três imagens por segundo exigem fechar cada ciclo em menos de 333 ms, e o Pi 4 leva 721.

Não é que ele seja lento: é aritmeticamente incapaz. O teto dele é 1,39 imagens por segundo.

E agora a tela mostra as duas taxas lado a lado, então isso não pode voltar a acontecer sem que apareça.

"Vamos ver o quanto isso pesa em processamento."

O detector é praticamente o ciclo inteiro nas duas placas. A projeção do raio, o rastreador e a camada de identidade **juntos não somam um milissegundo mensurável**.

Em termos de carga: pedindo meia imagem por segundo, o Pi 4 trabalha 36 % do tempo e o Pi 5 trabalha 5 %. É por isso que um esquenta e o outro não.

E o firmware do próprio Pi 4 confirma: ao desligar reportou `throttled=0xe0000`, que são três bits — limite térmico atingido, fredda ocorrida, frequência do ARM limitada. Não é dedução da curva, é a placa dizendo.

"O vetor de embedding muda muito dependendo da direção?"

Muda, e você tinha razão em desconfiar. Medido sobre vetores médios em janelas de quatro segundos: com o limiar que usamos, **35 % dos pares de pessoas DIFERENTES passam no filtro de aparência**.

Ou seja, a aparência sozinha não distingue pessoas. O que sustenta o sistema é que **a distância filtra primeiro**, e a aparência decide entre os que já estão perto. Aquilo que eu disse na reunião, "é os dois", estava certo — agora tem número.

O que isso proíbe é uma frase: não dá para dizer "a aparência distingue pessoas".

"Vamos trabalhar só com dois drones."

Combinado. E aí está o problema: **o Pi 4 é o que limita**. Não voa a 3 imagens por segundo, e a 1 ele cruza os 80 °C em oito minutos.

Mas tem uma coisa boa, e é contraintuitiva: medimos quanto custa baixar a taxa, e **não custa detecções**. Com um quarto das imagens o sistema acha as mesmas cinco pessoas de sete e produz **metade dos falsos positivos**.

Então o calor deixa de ser um impedimento e vira um parâmetro. Minha proposta é um dissipador no Pi 4, que custa cinco dólares, e voar os dois.

Si pregunta qué falta

Falta a energia por imagem, que é o que traduz taxa em minutos de autonomia, e o percentil 95 da latência — hoje a instrumentação está pronta mas a meia imagem por segundo entra uma amostra só por relatório, então o p95 é a mediana. Preciso rodar a 3 e 4 para isso ter sentido.

Anexo A. El núcleo del código

Cuatro piezas. No cuarenta.

A.1 Del píxel al rayo

```
def pixel_to_ray(pos, yaw_deg, pixel, pitch_deg, f_px, w, h, principal,
                 body_pitch_deg=0.0, body_roll_deg=0.0):
```

```

"""Un píxel y una pose dan una semirrecta en el mundo."""
cx, cy = principal
u, v = pixel
# Dirección en el sistema de la cámara. La focal va en z porque el plano
# imagen está a f píxeles del centro óptico.
d_cam = np.array([u - cx, v - cy, f_px], dtype=float)
d_cam /= np.linalg.norm(d_cam)
R = _world_to_camera_rotation(yaw_deg, pitch_deg, body_pitch_deg, body_roll_deg)
return np.asarray(pos, dtype=float), R.T @ d_cam

```

Por qué así. La focal entra como tercera componente, no como escala: el plano imagen está a `f` píxeles del centro óptico, y esa es la definición del modelo pinhole. La rotación se traspone porque `R` lleva del mundo a la cámara y acá se va en la dirección contraria.

A.2 Del rayo al suelo

```

def _ground_impact(origin, direction, ground_z):
    """Corta la semirrecta con el plano horizontal. None si no lo alcanza."""
    if direction[2] >= 0:          # apunta al horizonte o arriba
        return None
    k = (ground_z - origin[2]) / direction[2]
    if k <= 0:                    # el plano queda detrás de la cámara
        return None
    return origin + k * direction

```

Por qué las dos guardas. La primera descarta rayos que nunca cortan el plano. La segunda descarta el corte que existe matemáticamente pero está **detrás** de la cámara: una semirrecta no es una recta, y olvidarlo pone objetivos a espaldas del dron.

A.3 El radio de fusión

```

# fusion_radius_m = gps_sigma + slant_range * yaw_sigma
#
# Dos fuentes de error que no se comportan igual. La del GPS no depende de a
# dónde mire la aeronave. La de rumbo se multiplica por la distancia: un grado
# cuesta 0,75 m a 42 m de rango inclinado, y 1,7 m a 100 m.
self.yaw_sigma_rad = math.sqrt(max(0.0, bias_sigma_m**2 - gps_sigma_m**2)) /
    RANGO_REFERENCIA_M

```

Por qué una fórmula y no un número. Todos los términos están en la telemetría, así que el radio se recalcula por detección en vez de ajustarse a mano para un vuelo y quedar mal en el siguiente.

A.4 La instrumentación de latencia

```

_t0_ciclo = time.perf_counter()
_t0_det = time.perf_counter()
detecciones = self.camera.detect(self._position, yaw)
self._lat_detector.append((time.perf_counter() - _t0_det) * 1000.0)
...
self._lat_ciclo.append((time.perf_counter() - _t0_ciclo) * 1000.0)

```

```
def _latencias(self):
    """Lo que costó el lazo en el ÚLTIMO intervalo, y después lo olvida."""
    d = {"ciclo_p50": self._percentil(self._lat_ciclo, 0.50),
        "ciclo_p95": self._percentil(self._lat_ciclo, 0.95),
        "detector_p50": self._percentil(self._lat_detector, 0.50),
        "n": len(self._lat_ciclo)}
    self._lat_ciclo = []
    self._lat_detector = []
    return d
```

Tres decisiones en diez líneas. `perf_counter` y no el reloj del proveedor, porque en una reproducción ese reloj informaría lo que un cuadro *debía* tardar. Dos cronómetros anidados, porque el total no dice dónde está el costo. Y el vaciado al publicar, porque un percentil acumulado queda dominado por el arranque.

Anexo B. Cómo reproducir todo

```
# El barrido de tasas, sobre hardware
python scripts/medir/barrido_fps.py --estacion <ip:puerto> \
    --placas pi@<pi5> pi@<pi4> --tasas 0.5 1 2 3 4 --minutos 10

# La precisión por tasa, sin hardware
python scripts/replay_vuelo3.py --pistas=demo/data/pistas_bot_cmc_sof.npz \
    --preliminares --submuestreo=N --candidatos=salida.json
python scripts/personas_encontradas.py --candidatos salida.json
```

Dos avisos que cuestan una tarde si no se saben. Las pistas externas y `--preliminares` no son opcionales: con el seguidor interno del replay la línea base da 1 de 7 en vez de 5 de 7, y entonces nada de lo que sigue es comparable. Y diez minutos por punto no es prudencia: a los dos minutos la Pi 4 miente.

Datos crudos, registros de las dos placas y de la estación: `docs/medidas_9oct/`.